

# Car Agency Management System

*Project Specification and Requirements*

**Using AVL Tree, Queues, and Stacks**

**Due date: 9/9/2026 @11:00P.M**  
**SUBMISSION WILL BE @ITC-Meta Course**

A structured specification for implementation, testing, and demonstration

## 1. Project Overview

The Car Agency Management System is a data-structure-based application for managing customers, vehicles, maintenance requests, reservations, and financial transactions. The project is designed to demonstrate how an AVL tree, queues, and stacks can be selected because of the behavior they provide rather than because a data structure is required by name. The system should separate business entities from the data structures that manage them. The final application must provide a clear JavaFX interface through which an agency employee can perform the required operations. Students must implement the required data structures rather than relying on the built-in collection classes for the core project.

## 2. Project Goals

1. Manage customers and their information.
  2. Store and manage the agency vehicle inventory in an AVL tree.
  3. Process maintenance requests in arrival order using a queue.
  4. Manage customer waiting requests using a queue.
  5. Support vehicle reservations using a queue when a vehicle is unavailable.
  6. Maintain recent service history using a stack where LIFO behavior is meaningful.
  7. Support undo for selected reversible operations using a stack.
  8. Record sales and service-related financial transactions.
  9. Demonstrate correct handling of empty structures, duplicates, invalid values, and unavailable vehicles.
  10. Analyze the time complexity of the implemented data structures and major operations.
  11. **Read files (all files need it for this project) using file chooser.**
- Stack, Queue must be implemented using linked lists. **Using any other data structure will result in a grade of ZERO for the entire project.** For Example, using Java's built-in `LinkedList`, `ArrayList`, or other collection classes **are not allowed.** (Scene Builder is not allowed).

### 3. Actors

The project has the following primary actors. Each actor interacts with the system for a clearly defined purpose.

| Actor                    | Role in the System      | Main Responsibilities                                                                                                      |
|--------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Agency Employee          | Primary system operator | Manage customers, vehicles, transactions, service requests, queues, reservations, and supported undo operations.           |
| Customer                 | Requester / participant | Provide customer information, request vehicle service, wait for service or sales assistance, and request/reserve vehicles. |
| Mechanic / Service Staff | Service processor       | Process maintenance requests from the maintenance queue, complete services, and update service status/history.             |

Note: Student can deal with all actors from same interface (application).

### 4. Data Structures and Their Responsibilities

| Data Structure | Mandatory Use          | Why It Fits                                                                                                          |
|----------------|------------------------|----------------------------------------------------------------------------------------------------------------------|
| AVL Tree       | Vehicle inventory      | Keeps vehicles ordered by unique vehicleId and supports balanced search, insertion, deletion, and ordered traversal. |
| Queue<T>       | Maintenance queue      | Processes service requests in FIFO order according to arrival time.                                                  |
| Queue<T>       | Customer waiting queue | Processes waiting customers fairly in FIFO order.                                                                    |
| Queue<T>       | Reservation queue      | Processes customers waiting for a specific unavailable vehicle in FIFO order.                                        |
| Stack<T>       | Undo stack             | Reverses the most recent supported operation first (LIFO).                                                           |
| Stack<T>       | Service-history stack  | Provides fast access to the most recently recorded service.                                                          |
| Stack<T>       | Redo stack             | Restores an operation that was previously undone, using a second stack.                                              |

Students must implement the core Stack<T> and Queue<T> classes. The project should not depend entirely on built-in collection classes for these structures.

## 5. Main System Entities (Classes)

| Class          | Purpose                                                   | Required / Suggested Fields                                               |
|----------------|-----------------------------------------------------------|---------------------------------------------------------------------------|
| Customer       | Represents a customer of the agency.                      | customerId, name, phone, address                                          |
| Vehicle        | Represents a vehicle managed by the agency.               | vehicleId, make, model, year, price, color, status                        |
| ServiceRequest | Represents a maintenance request waiting to be processed. | requestId/serviceId, vehicleId, customerId, serviceType, date, status     |
| Service        | Represents a completed maintenance record.                | serviceId, vehicleId, customerId, serviceType, date, cost, status         |
| Transaction    | Represents a financial operation.                         | transactionId, customerId, vehicleId, amount, transactionType, date       |
| AVLNode        | Represents a node in the vehicle AVL tree.                | Vehicle object, key/vehicleId, left, right, height/balance information    |
| Stack<T>       | Generic LIFO structure.                                   | push, pop, peek, isEmpty, size; additional methods as needed              |
| Queue<T>       | Generic FIFO structure.                                   | enqueue, dequeue, peek/front, isEmpty, size; additional methods as needed |

Clarification: The source project refers to both Service and ServiceRequest. In this specification, ServiceRequest is the queued request, while Service is the completed service record. This distinction avoids duplication of responsibilities.

**Files :** files are separated by “,”. These files should be read by file chooser.

|                |
|----------------|
| Customer       |
| Vehicle        |
| ServiceRequest |
| Service        |
| Transaction    |

## 6. Vehicle Status Values

| Status     | Meaning                                                                              |
|------------|--------------------------------------------------------------------------------------|
| AVAILABLE  | Vehicle can normally be sold/reserved according to agency rules.                     |
| SOLD       | Vehicle has been sold and cannot be sold again.                                      |
| IN_SERVICE | Vehicle is currently undergoing maintenance and cannot normally be sold or reserved. |
| RESERVED   | Vehicle is currently reserved according to the reservation process.                  |

## 7. Complete Required Operations

This section contains the complete set of operations that the student application must support. Operations are grouped by module and ordered by implementation priority. **(Use Tabs, TableViews, Stages and buttons in the proper places, you have to maintain the logic).**

### 7.1 Customer Operations

1. Add a new customer.
2. Search for a customer by customerId.
3. Display customer information.
4. Update customer information.
5. Delete a customer.

### 7.2 Vehicle / AVL Operations

1. Add a vehicle to the AVL tree using the unique vehicleId as the key.
2. Search for a vehicle by vehicleId.
3. Display vehicles in ascending order using AVL in-order traversal.
4. Display vehicles in descending order using reverse in-order traversal.
5. Update vehicle information without changing the AVL key.
6. Change the vehicle status.
7. Mark a vehicle as SOLD.
8. Mark a vehicle as IN\_SERVICE when maintenance starts.
9. Mark a vehicle as AVAILABLE when maintenance is completed.
10. Delete a vehicle.
11. Print the high of tree.

### 7.3 Maintenance Service Operations

1. Create a service request for a customer and vehicle.
2. Validate that the requested vehicle exists.
3. Add the service request to the maintenance queue.
4. Display all pending service requests in queue order.
5. Process the next service request using dequeue() and FIFO order.
6. Change the vehicle status from AVAILABLE to IN\_SERVICE when processing starts.
7. Complete the service and record the service details and cost.
8. Create/store a completed Service record.
9. Push the completed service record onto the vehicle service-history stack.
10. Change the vehicle status from IN\_SERVICE back to AVAILABLE after completion.

### 7.4 Customer Waiting Queue Operations

1. Add a customer to the waiting queue.
2. Display the waiting queue from front to rear.
3. Display the number of waiting customers.
4. Serve/remove the next customer using dequeue().
5. Maintain FIFO order: the earliest waiting customer must be served first.

### 7.5 Vehicle Reservation Queue Operations

1. Allow a customer to request a vehicle that is currently unavailable.
2. Add the request to the reservation queue for that vehicle.
3. Display reservation requests in FIFO order.
4. When the vehicle becomes available, process the customer at the front of the queue first.

### 7.6 Transaction Operations

1. Create a purchase/sale transaction.
2. Create a service-payment transaction.
3. Record a deposit.
4. Record a refund when permitted.

5. Display transaction information.
6. Validate that transaction amounts, vehicle prices, and service costs are not negative unless the operation explicitly represents a refund or adjustment.

### 7.7 Undo Stack Operations

1. Record each supported reversible operation by pushing an operation object onto the Undo Stack.
2. Undo the most recently performed supported operation using pop().
3. Reverse the operation through its undo behavior.
4. Do not assume every operation is reversible; external or completed financial operations may require special handling.
5. Handle an empty Undo Stack safely.

### 7.8 Service-History Stack Operations

1. Push each completed service record onto the vehicle history stack.
2. Display the most recently completed service first using peek() or equivalent.
3. Support display of recent service history in LIFO order.
4. Do not use the stack as the primary structure for arbitrary historical searching.

### 7.9 Required Generic Data-Structure Operations

| Structure | Minimum Operations                                                                                             |
|-----------|----------------------------------------------------------------------------------------------------------------|
| Stack<T>  | push(T), pop(), peek(), isEmpty(), size()                                                                      |
| Queue<T>  | enqueue(T), dequeue(), peek()/front(), isEmpty(), size()                                                       |
| AVL Tree  | insert, search, delete, rotations, height/balance maintenance, in-order traversal, reverse in-order traversal. |

## 8. System Workflow

The following workflow shows how the modules cooperate. The same sequence should be reflected in the implementation and demonstrated during project evaluation.

1. The employee searches the AVL tree for the vehicle using vehicleId.
2. The employee verifies the vehicle status.
3. A customer creates a maintenance request.
4. The request is placed at the rear of the maintenance queue.
5. The mechanic processes the request at the front of the queue.
6. The vehicle status changes to IN\_SERVICE.
7. When the work is completed, a Service record is created.
8. The completed Service record is pushed onto the service-history stack.
9. The vehicle status changes to AVAILABLE.
10. Any supported reversible employee operation may be recorded on the Undo Stack.

## 9. Priority Classification

To make the project requirements unambiguous, every requirement is classified as Mandatory (P0/P1).

| Priority | Requirement                                                                                                         | Status    |
|----------|---------------------------------------------------------------------------------------------------------------------|-----------|
| P0       | Implement Customer, Vehicle, ServiceRequest, Service, Transaction, AVLNode, generic Stack<T>, and generic Queue<T>. | Mandatory |
| P0       | Implement AVL vehicle insertion, search, deletion, balancing, and ordered traversal.                                | Mandatory |

| Priority | Requirement                                                                                | Status    |
|----------|--------------------------------------------------------------------------------------------|-----------|
| P0       | Implement customer management operations.                                                  | Mandatory |
| P0       | Implement vehicle management operations.                                                   | Mandatory |
| P0       | Implement maintenance queue and FIFO processing.                                           | Mandatory |
| P0       | Implement customer waiting queue and FIFO processing.                                      | Mandatory |
| P0       | Implement transaction recording.                                                           | Mandatory |
| P1       | Implement Undo Stack for selected reversible operations.                                   | Mandatory |
| P1       | Implement service-history stack.                                                           | Mandatory |
| P1       | Handle empty stack/queue, duplicate IDs, invalid values, and vehicle-status rules.         | Mandatory |
| P1       | Provide JavaFX interface for the required operations.                                      | Mandatory |
| P1       | Provide demonstration of the complete integrated workflow.                                 | Mandatory |
| P1       | Provide time-complexity analysis for the implemented data structures and major operations. | Mandatory |
| P1       | Redo using a second stack.                                                                 | Mandatory |

## 10. User Interface Requirements

The final application should provide a clear JavaFX interface organized into functional tabs or panels. The following organization is recommended:

| Tab / Panel            | Required Functions                                                                     |
|------------------------|----------------------------------------------------------------------------------------|
| Customer Management    | Add, search, display, update, delete.                                                  |
| Vehicle Management     | Add, search, update, delete, change status, mark sold, display ascending/descending.   |
| Maintenance            | Create request, display queue, process next request, complete service, record history. |
| Customer Waiting Queue | Enqueue, display, serve next, count waiting customers.                                 |
| Reservations           | Add reservation, display queue, process next eligible reservation.                     |
| Transactions           | Create sale, service payment, deposit, refund, display transaction.                    |
| Undo / Redo            | Undo supported operation; Redo supported operations.                                   |

## 11. Required Demonstration Scenarios (Samples)

### Scenario 1 – AVL Vehicle Search

Insert vehicles with IDs 50, 30, 70, 20, 40, 60, and 80. Search for vehicle 60 and display its details. Demonstrate that the vehicle remains in the AVL tree when only its status changes.

### Scenario 2 – Customer Waiting Queue

Enqueue Ahmed, Sara, and Omar. Dequeue once. Ahmed must be removed first, leaving Sara followed by Omar.

### Scenario 3 – Maintenance Queue

Enqueue three service requests: Toyota Corolla – Oil Change; Honda Civic – Brake Service; Kia Sportage – Tire Replacement. Process them in exactly the same arrival order.

### Scenario 4 – Undo Stack

Perform: Add Customer → Add Vehicle → Create Transaction → Apply Discount. Undo once. The Apply Discount operation must be reversed first.

### Scenario 5 – Complete Integrated Workflow

A customer requests maintenance for vehicle 60. Search the AVL tree, verify availability, create the service request, enqueue it, dequeue it for processing, change the vehicle status to IN\_SERVICE, complete the service, push the completed record onto the service-history stack, and finally change the vehicle status to AVAILABLE.

## 12. Error and Edge-Case Handling

Handle all expected errors in this project.

# Original Work

All submitted work must be the student's own work. Students are expected to understand and be able to explain every part of their implementation. Copying code from other students, previous semesters, or online sources/AI tools is not permitted and surly the project will NOT BE GRADED.